# Hostel Management System

Database Design — v4 (Meal separated from Room, OTP moved out of DB, design fixes)

---

## Summary of Changes from v2

• Removed **mealsIncluded** and **mealPlan** fields from Room Collection.

• Added new **Meal Plan Collection** (master catalog of meal plans).

• Added new **User Meal Subscription Collection** (links a user to a meal plan).

• Fee Collection updated with **feePeriod** for recurring (monthly/semester) billing.

• Booking Collection updated with **checkInDate** / **checkOutDate**.

• Removed **OTP Collection** from the DB design — OTPs should live in a short-lived store (Redis TTL, or a MongoDB TTL-indexed collection) rather than the primary database.

• Admin Logs Collection updated with **targetType** / **targetId** for structured auditing.

• Room's **occupiedCount** flagged as a derived/cache field — keep it in sync via booking status changes, or compute on demand.

# 1. Core Collections

## 1.1 User Collection

```
{
_id,
name,
email,
password,
phone,
gender,
courseId,
address,
role,
isActive,
createdAt,
updatedAt,
}
```

## 1.2 Course Collection

```
{
_id,
courseName,
courseCode,
duration,
createdAt,
}
```

## 1.3 Room Collection (cleaned up)

Meal-related fields removed — Room now only represents physical space.

```
{
_id,
roomNumber,
seater,
fees,
occupiedCount,
isAvailable,
createdAt,
updatedAt, ← NEW/CHANGED
}
```

## 1.4 Booking Collection (updated)

```
{
_id,
userId,
roomId,
bookingDate,
checkInDate, ← NEW/CHANGED
checkOutDate, ← NEW/CHANGED
status,
createdAt,
}
```

# 2. Meal Collections (New)

Meals are now fully decoupled from Room. This lets a student change their meal plan, pause meals, or opt out entirely without touching the room record, and supports meal-specific pricing and multiple plan tiers.

## 2.1 Meal Plan Collection (NEW)

Master catalog of meal plans, managed by admin.

```
{
_id,
planName,
type,
price,
mealsPerDay,
description,
isActive,
createdAt,
}
```

## 2.2 User Meal Subscription Collection (NEW)

Links a user to a meal plan; tracks their subscription lifecycle.

```
{
_id,
userId,
mealPlanId,
startDate,
endDate,
status,
createdAt,
updatedAt,
}
```

# 3. Fees & Payments

## 3.1 Fee Collection (updated)

Added feePeriod to support recurring monthly/semester billing cycles.

```
{
_id,
userId,
roomId,
mealSubscriptionId, ← NEW/CHANGED
feePeriod, ← NEW/CHANGED
totalFees,
paidAmount,
dueAmount,
paymentStatus,
dueDate,
createdAt,
updatedAt,
}
```

## 3.2 Payment Collection

```
{
_id,
userId,
feeId,
amount,
paymentMethod,
transactionId,
paymentDate,
status,
createdAt,
}
```

# 4. Attendance

## 4.1 Attendance Collection

```
{
_id,
userId,
date,
status,
markedBy,
createdAt,
}
```

## 4.2 Attendance Summary Collection

Treat as a computed rollup/cache, regenerated by a scheduled job — not written to directly.

```
{
_id,
userId,
month,
year,
```

```
totalDays,
presentDays,
absentDays,
}
```

# 5. Support, Feedback & Admin

## 5.1 Complaint Collection

```
{
_id,
userId,
title,
description,
status,
adminResponse,
createdAt,
updatedAt,
}
```

## 5.2 Feedback Collection

```
{
_id,
userId,
message,
rating,
createdAt,
}
```

## 5.3 Admin Logs Collection (updated)

Added targetType / targetId for structured auditing and filtering.

```
{
_id,
adminId,
action,
description,
targetType, ← NEW/CHANGED
targetId, ← NEW/CHANGED
createdAt,
}
```

## 6. Key Relationships

| Collection | References | Relationship |
|---|---|---|
| User | Course | courseId → Course._id |
| Booking | User, Room | userId, roomId |
| Fee | User, Room, User Meal Subscription | userId, roomId, mealSubscriptionId |
| Payment | User, Fee | userId, feeId |
| Attendance | User | userId |
| Attendance Summary | User | userId (rollup of Attendance) |
| Complaint | User | userId |
| Feedback | User | userId |
| User Meal Subscription | User, Meal Plan | userId, mealPlanId |
| Admin Logs | Admin (User), Target Entity | adminId, targetType + targetId |

## 7. Recommended Indexes

• userId — index in Booking, Fee, Payment, Attendance, Complaint, Feedback, User Meal Subscription

• roomId — index in Booking, Fee

• email — unique index in User, index in OTP

• mealPlanId — index in User Meal Subscription

• feePeriod — index in Fee (for monthly billing queries)

• targetType + targetId — compound index in Admin Logs

---

Note: occupiedCount in Room is a denormalized counter. Recommend either computing it on-demand from active Booking records, or updating it transactionally whenever a booking's status changes to keep it from drifting out of sync.

Note: OTP Collection has been removed from this design. OTPs are short-lived and don't belong in the primary database long-term. Recommended approach: use an in-memory store such as Redis with native key TTL, or if staying on MongoDB only, a separate collection with a TTL index on expiresAt (expireAfterSeconds: 0) so expired OTPs self-delete automatically.